

Apresentação de Slides

Modelagem Informacional

Prof. Dr. Wagner Perin

FUCAPE Business School

Modelagem para Sistemas NoSQL e Persistência Poliglota

A Revolução Além das Tabelas: Documentos, Grafos e Chave-Valor

Disciplina: Modelagem Informacional

Programa: Ciência de Dados para Negócios | **Carga Horária:** 4 h/a

Professor: Prof. Dr. Wagner Perin

Recapitulando: Onde estamos na Jornada de Dados?

- **Aulas 03 e 04:** Dominamos o modelo relacional clássico (OLTP) em 3ª Forma Normal (3FN), garantindo integridade transacional estrita (ACID).
- **Aula 05:** Aprendemos a desnormalizar dados para análise corporativa massiva com a Modelagem Multidimensional (Star Schema/Data Warehouse).

*Mas e quando os dados **não se encaixam confortavelmente em tabelas bidimensionais**? E quando possuem esquemas dinâmicos, são altamente conectados, ou exigem latência de milissegundos?*

Hoje: Entenderemos o ecossistema NoSQL (*Not Only SQL*) e a estratégia corporativa de **Polyglot Persistence**.

Módulo 1: Abertura e Dor de Negócio

O Colapso do Monolito e os Limites do Modelo Relacional

O Colapso da Black Friday na OmniMarketplace

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Problema Investigado: A OmniMarketplace, gigante do e-commerce com 5 milhões de itens, enfrentou desastres simultâneos na Black Friday, causando R\$ 4,2 milhões em prejuízos diretos ao forçar um único banco relacional para toda a operação.



Painel de Operações da OmniMarketplace na Black Friday

As Três Dores da OmniMarketplace

1. O Pesadelo do Catálogo Heterogêneo (Antipadrão EAV):

- Produtos distintos (smartphones, pneus, vestidos) forçaram a criação de uma tabela Entity-Attribute-Value com 120 milhões de linhas. Filtros simultâneos na busca geraram dezenas de auto-JOINS. CPU a 100%. **Buscas levando 28 segundos.**

2. O Travamento das Recomendações em Tempo Real:

- Uma query recursiva de 4 graus ("amigos que compraram") cruzou milhões de compras históricas. O banco relacional entrou em *starvation* e derrubou o checkout.

3. Abandono em Massa de Carrinhos:

- Gravar o carrinho efêmero no disco rígido do banco relacional gerou concorrência de I/O insustentável. **Erro 504: Gateway Timeout.**

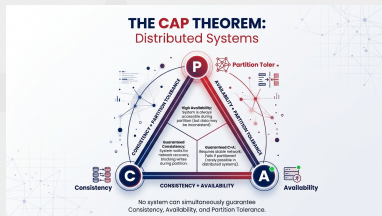
O Movimento NoSQL (*Not Only SQL*)

Não se trata de rejeitar o SQL, mas de expandir o repertório para problemas onde **tabelas rígidas e transações ACID custam muito caro**.

Por que focar em NoSQL agora?

- **Flexibilidade Estrutural:** Lidar com dados semiestruturados, variados e mutáveis (*Schema-on-Read*).
- **Escalabilidade Horizontal:** Escalar distribuindo dados por múltiplos nós mais baratos, em vez de um único super-servidor caro (Escala Vertical).
- **Adequação ao Domínio:** Usar a estrutura de dados nativa (grafos, documentos) para resolver o problema de forma natural e performática.

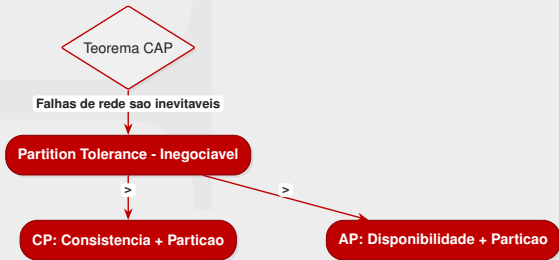
O Teorema CAP de Eric Brewer



Os Três Vértices do Teorema CAP

Regra Fundamental: Em sistemas distribuídos, é impossível garantir simultaneamente Consistência (C), Alta Disponibilidade (A) e Tolerância a Particionamento (P).

O Teorema CAP na Computação em Nuvem



A Verdade Dura: Em nuvens públicas (AWS, GCP, Azure), redes físicas falham rotineiramente. Portanto, **P é inegociável**. A verdadeira decisão de arquitetura é entre **CP ou AP**.

Dilema CP vs AP no Mundo Real

Sistemas CP (Consistência Forte)

- *Exemplo:* MongoDB (com quorum de escrita estrito), Bancos Relacionais Distribuídos.
- *Comportamento:* Se a rede partir, o sistema rejeita novas requisições para evitar leituras inconsistentes. A consistência é preservada ao custo da disponibilidade.

Sistemas AP (Consistência Eventual)

- *Exemplo:* Apache Cassandra, Amazon DynamoDB, CouchDB.
- *Comportamento:* Se a rede partir, qualquer nó responde imediatamente com o dado que possui. Adota-se o princípio da **Consistência Eventual** (*Eventual Consistency*).

Trade-off de Negócios: Consistência vs.

Disponibilidade

Decisão Executiva sob Pressão

Desafio Corporativo & Questões de Impacto: "Você é o Diretor de Tecnologia da OmniMarketplace nas 2 últimas horas da Black Friday. Você prefere que o botão 'Comprar' dê Erro 500 no site (CP - Consistência Forte) ou que ele conclua a venda com risco mínimo de vender um item que acabou há 3 segundos (AP - Consistência Eventual) e compensar o cliente depois com um voucher?"

Reflexão:

- Como a teoria matemática de Brewer se traduz em cálculo de risco, margem financeira e experiência do cliente?
- A consistência estrita vale o custo do abandono imediato de carrinho?

Módulo 2: As Três Famílias NoSQL

Documentos (MongoDB), Grafos (Neo4j) e Chave-Valor (Redis)

Família 1: Bancos Orientados a Documentos (MongoDB)

O Fim do Antipadrão EAV.

- A unidade atômica é um documento **JSON/BSON**, não uma linha em uma tabela.
- **Schema-on-Read:** A estrutura não é forçada na gravação. Documentos da mesma coleção podem possuir atributos completamente distintos.
- **Hierarquia Rica:** Permite aninhar dados (*arrays* e subdocumentos) em um só lugar.

JSON – Código Fonte

```
{
  "_id": "tv4k_909",
  "sku": "TV-4K-65",
  "nome": "Smart TV 65 polegadas",
  "especificacoes": {
    "voltagem": "Bivolt",
    "frequencia_hz": 120
  }
}
```

O Grande Dilema de Modelagem em Documentos

Como os dados devem ser estruturados?

1. Documentos Embutidos (*Embedded Documents* / *Denormalization*):

- **Vantagem:** Leitura atômica em 1 operação de I/O (*Zero JOINS*). Alta localidade.
- **Uso:** Relações 1:1 ou 1:Poucos (Ex: Produto → Especificações Técnicas).
- **Risco:** Limite de 16 MB por documento no MongoDB. Duplicação de dados.

2. Referências Normalizadas (*Referencing* / *Normalization*):

- **Vantagem:** Evita duplicação e documentos gigantescos.
- **Uso:** Relações 1:Muitos ilimitados ou M:N (Ex: Produto → Milhões de Avaliações).
- **Risco:** Exige múltiplas consultas ou operadores tipo `$lookup` na aplicação.

Família 2: Bancos Orientados a Grafos (Neo4j)



Rede de Conexões e Relacionamentos em Grafos

Mudança de Paradigma: No modelo relacional, JOINS são computados em tempo de query. Em Grafos, o relacionamento é cidadão de primeira classe gravado fisicamente no disco.

Anatomia do Property Graph Model

Elementos Fundamentais:

- **Nós (*Nodes*):** Entidades com rótulos (Ex: :Cliente, :Produto, :Categoria).
- **Arestas (*Relationships*):** Conexões direcionadas, tipadas e com propriedades (Ex: -[:COMPROU {data: '2026-09-01'}]->).
- **Propriedades:** Pares chave-valor armazenados em nós e arestas.

O Segredo de Performance: *Index-Free Adjacency*

- No modelo relacional, um JOIN pesado exige varrer índices globais em Árvore-B ($O(\log N)$).
- Em grafos com **Index-Free Adjacency**, cada nó armazena **ponteiros físicos diretos** para seus vizinhos.
- A travessia de relacionamentos possui complexidade de tempo $O(1)$ por nó visitado, mesmo com bilhões de nós no banco!

Exemplo Cypher: O Poder do Grafo

A recomendação em tempo real que derrubou a *OmniMarketplace*, resolvida em milissegundos:

SQL – Código Fonte

```
-- Cypher Query: Quem comprou este mesmo produto, o que mais comprou?  
MATCH (c1:Cliente)-[:COMPROU]->(p1:Produto {sku: 'TV-4K'})  
MATCH (c2:Cliente)-[:COMPROU]->(p1)  
MATCH (c2)-[:COMPROU]->(p2:Produto)  
WHERE c1 <> c2 AND NOT (c1)-[:COMPROU]->(p2)  
RETURN p2.nome, count(p2) AS frequencia  
ORDER BY frequencia DESC LIMIT 5;
```

Resultado: O banco apenas caminha pelos ponteiros de memória locais. Ele não varre a tabela global de compras!

Família 3: Bancos Chave-Valor (Redis)

Gerenciamento de Carrinho Efêmero com TTL de 2 horas (7200s)

- Um dicionário gigante processado **estritamente em memória RAM**.
- **Performance Extrema:** Latência de submilissegundo para leituras e escritas.
- **TTL (Time-To-Live):** Descarte automático de dados voláteis sem rotinas noturnas de limpeza (DELETE). A chave se autodestrói quando o prazo expira.
- **Operações Atômicas:** Incrementos em tempo real (INCR) sem *locks* de tabela.

Bash – Código Fonte

```
HSET cart:user_8899 "TV-4K-65" 1
HSET cart:user_8899 "SOUNDBAR-90" 1
EXPIRE cart:user_8899 7200
```

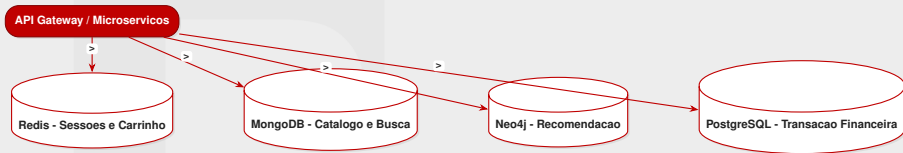
Trade-off Fundamental: Memória RAM é cara. Redis é para **dados quentes (Hot Data)** e **efêmeros**, não para histórico contábil!

Módulo 3: Desafio Prático de Código (PBL)

Redesenho da Arquitetura Poliglota da OmniMarketplace

O Desafio: Redesenho da Arquitetura Poliglota

Vocês agora compõem o **Comitê de Arquitetura de Dados** da OmniMarketplace. O monolito relacional falhou. Precisamos desenhar a solução utilizando a tecnologia correta para cada domínio.



Polyglot Persistence: Usar múltiplas tecnologias de banco de dados na mesma organização, de acordo com o problema a ser resolvido.

Tarefas do Desafio Prático (PBL)

- **Tarefa 1 (MongoDB) — Modelar o Catálogo Heterogêneo:** Criar o JSON para produtos heterogêneos (TV, Camiseta, Pneu). Decidir o que é embutido e o que é referenciado, eliminando o modelo EAV.
- **Tarefa 2 (Neo4j) — O Motor de Recomendação:** Desenhar o modelo de Grafo em Mermaid.js especificando Nós (Clientes/Produtos) e Arestas. Explicar a travessia com *Index-Free Adjacency* em $O(1)$.
- **Tarefa 3 (Redis) — Estratégia de Carrinho Efêmero:** Definir o padrão de chave (`cart:{id}`), TTL (2h) e calcular a memória RAM para 200.000 carrinhos simultâneos (500 bytes por carrinho $\approx$ 95 MB).

Módulo 4: Tomada de Decisão & Síntese

Matriz de Persistência Poliglota e Teaser da Aula 07

Pitch de Arquitetura: Validação Executiva

Apresentação das decisões técnicas pelas equipes.

Dilemas de Arquitetura & Decisão Analítica:

1. Se o MongoDB sofrer partição de rede na escrita, o novo produto fica invisível (AP) ou a escrita falha (CP)? O que foi configurado?
2. Se o Redis usa RAM e é tão rápido, por que não o usamos para guardar todo o faturamento dos últimos 5 anos?
3. Como garantiremos que o preço no PostgreSQL, MongoDB e Redis seja atualizado sem gerar dados fantasmas?

A Matriz de Decisão Arquitetural (Polyglot)

Tecnologia	Complexidade I/O	Ponto Forte	Quando NÃO Usar
Relacional (RDBMS)	$O(\log N)$	ACID, transações financeiras e contábeis.	Catálogos heterogêneos, Big Data não-estruturado.
Documentos (MongoDB)	$O(1)$ no ID	Catálogos flexíveis, CMS, objetos aninhados ricos.	Relacionamentos profundos exigindo centenas de JOINS transversais.
Grafos (Neo4j)	$O(1)$ por aresta	Motores de recomendação, linhagem, prevenção a fraudes.	Relatórios analíticos massivos (varredura agregada global).
Chave-Valor (Redis)	$O(1)$ em RAM	Sessões, carrinhos efêmeros, rate limiting, leaderboards.	Consultas complexas usando atributos internos (WHERE nome=X).

Síntese da Aula & Takeaways Executivos

3 Lições Fundamentais:

1. **Polyglot Persistence é o padrão:** O monolito morreu em alta escala. Bancos são ferramentas especializadas, não canivetes suíços.
2. **Modelagem Orientada a Acesso:** No MongoDB, você modela em função de *como a aplicação fará a leitura*, equilibrando Embutimento vs Referências.
3. **O Grafo domina a Conectividade:** Ponteiros físicos (Index-Free Adjacency) viabilizam consultas sociais e antifraude impossíveis no relacional clássico.

Teaser para a Próxima Aula: Engenharia de Pipelines

*"Hoje temos bancos relacionais, documentos MongoDB, grafos Neo4j e logs em nuvem...
Todos separados."*

A Pergunta de Milhões: Como extraímos, cruzamos e limpamos tudo isso para gerar Inteligência de Negócios Centralizada no Data Warehouse *sem derrubar as operações na Black Friday?*

Na Aula 07: Entraremos na Engenharia de Integração de Dados! O duelo **ETL vs. ELT**, orquestração distribuída com **Apache Airflow** e transformações modernas com **dbt**.